

LingBot-VLA 2.0 推理与训练加速改造: 从 1 秒级到 300 ms 级的全栈优化

编译友好的稠密 MoE、前缀编译与 GPU 预处理的测量、方法与对照实验

LingBot-VLA 加速工作组 测试平台: 阿里云 DSW 8×NVIDIA A100-SXM-80GB 代码分支
pr3967-combined(基于 lerobot PR #3967)

摘要 LingBot-VLA 2.0(lerobot PR #3967) 以 Qwen3-VL-4B 为视觉-语言骨干, 并联 36 层稀疏 MoE(32 experts、top-4) 动作专家, 流匹配 10 步去噪、每 chunk 输出 50 步动作。其 PR 原版在 A100 上单次推理 e2e 1039.5 ms: 分段计时表明去噪循环占 86.5%, 且为典型 launch-bound 工况 (51 token × 36 层双流, 每次推理约 360 次 MoE kernel launch); 另有 107-141 ms 预处理全在 CPU。本文报告 pr3967-combined 分支上六项按序落地的优化: sdpa+bf16 attention 与去噪循环不变量外提、torch.compile 去噪主循环、稠密双 GEMM 纯 PyTorch MoE(把 top-k 路由代数等价于两个普通 GEMM, 消除每层 graph break)、max-autotune 编译、预处理 GPU 化(复刻 HF processor 的 torch 算子序列、跳过 Python 包装)与 prefix 整段编译。A100 同机实测: SO101 单相机负载下 model-only 284.0 ms、最终栈 e2e 约 **300-315 ms**(较 PR 基线 model-only -72%、e2e -79%); 在官方 robotwin 三相机负载上与官方 deploy 同窗口对比, e2e 436.6 ms, 为官方 eager 的 **3.9×**、官方 use_compile 的 **2.55×**。训练侧 PR 单卡实现较官方 fsdp2 recipe 每卡吞吐约 **3.5×**。全部优化经 LIBERO 真实帧 open-loop drift 校验 ($\leq 2\%$ 阈值); fp16 方案因无收益且 drift 3.33% 超标被否决。

关键词: 视觉-语言-动作模型; 流匹配; Mixture-of-Experts; kernel 发射开销; torch.compile; GPU 预处理

1 实验基准

1.1 硬件、模型与负载

硬件。 阿里云 DSW 实例, 8×NVIDIA A100-SXM-80GB; 与 LingBot-VLA 2.0(6B) 官方训练/推理同机型。对照对象为官方 upstream 声明的 130 ms @ 4090D(10 步去噪)。

模型。 LingBot-VLA 2.0(lerobot PR #3967): Qwen3-VL-4B backbone + 36 层稀疏 MoE 动作专家 (32 experts、top-4 路由、专家中间维 512, 另有共享专家 704, hidden 768), 动作经 flow matching 生成, 10 步 Euler 去噪, 每 chunk 输出 50 步动作 (suffix 51 token: state 1 + action 50)。

负载。 两类: (a) SO101 单相机 ckpt(PR 侧主负载); (b) 官方 robotwin ckpt, 3 相机 480×640, 用于与官方 upstream deploy 的同权重同 workload 对比。

1.2 测量方法

自研 bench/bench_lingbot_v2.py: 分段 CUDA-event 计时 (vision / prefix / denoise / attention / moe / rope), e2e = preprocess + model + unapply。保真度用 LIBERO 真实帧 open-loop drift: 与 fp32/eager 参考的相对 L2(固定 noise 种子), 阈值 2%。

环境说明 (重要)。测试期间宿主机有他人 LIBERO eval 任务运行 (load 21–57)。本文所有对比数字均为同环境同窗口测量, 相对比较公平有效; 绝对值在空闲机上会更低。

2 基线与瓶颈分析

2.1 PR 原版

PR 原版 (eager attention + fp32) 在 A100、SO101 单相机 ckpt 下 e2e **1039.5 ms**。分段归因:

- 去噪循环占总时长 **86.5%**, 为典型 launch-bound: 51 token $\times$ 36 层双流结构, 每次推理约 360 次 MoE 调用, 单次 MoE kernel 仅 ~ 1 ms, 墙钟却被发射开销主导;
- preprocess 107–141 ms 全部在 CPU, 与 GPU 计算完全串行。

2.2 官方 upstream deploy

在 A100 上以官方 robotwin 三相机 (480 $\times$ 640) 负载实测官方 deploy 路径:

- eager:e2e **2284 ms**, 其中 CPU 预处理 1156 ms, 是最大的单项开销;
- use_compile:e2e 1567 ms;
- 官方 130 ms @4090D 的 claim 无法在 A100 上复现: 同机最优 compile model-only 为 344 ms。

2.3 官方训练 recipe

官方训练 recipe 在单张 A100 80GB 上**不可运行**: muon 优化器的 fp32 master 权重 + 动量 + 梯度固定占用约 74 GB; micro_batch=2 加 gradient checkpointing 仍 OOM。官方设计为 32 卡 fsdp2 分片, 单卡不在其支持包线内。

3 优化方案

六项优化按提交顺序全部落在 pr3967-combined 分支, 可逐项开关。

3.1 C1:sdpa + bf16 attention、去噪循环不变量外提与 sync-free MoE(commit 4d692d25)

1039.5 $\rightarrow$ **951.9 ms**。attention 切到 sdpa + bf16; 把去噪循环内每步重复构造的 masks、position_ids、mrope cos-sin 与 BlockMask 外提为循环不变量; Triton MoE 中去除 counts.max().item() 这一 host 同步点, 消除每步的 GPU-CPU 往返。

3.2 C2:torch.compile predict_velocity(commit b0544e9b)

$\rightarrow$ **699.5 ms**(-32.7%)。对 10 步去噪主循环 predict_velocity 经 inductor 编译 (cudagraphs 关闭)。

3.3 稠密双 GEMM 纯 PyTorch MoE(commit 9de3cc41, 核心创新)

不做路由, 用 2 个普通 GEMM 算完全部 32 个专家。设 token 数 T 、隐藏维 H 、专家数 E 、专家中间维 I : 把全部专家的 gate/up 权重按专家维拼接缓存为 $W_1 \in \mathbb{R}^{H \times E \cdot 2I}$, down 权重按输入维

permute 为 $W_2 \in \mathbb{R}^{E \cdot I \times H}$ 。前向:

$$[G \parallel U] = XW_1, \quad M = \text{SiLU}(G) \odot U \in \mathbb{R}^{T \times E \times I} \quad (1)$$

$$Y = \text{reshape}(M \odot R, [T, E \cdot I]) \cdot W_2 \quad (2)$$

其中 R 是把 top- k 路由权重 one-hot 回稠密表 $[T, E]$ 的结果。down GEMM 沿 $E \cdot I$ 维的归约天然完成“按权重合并专家”: 未选中的专家权重恒为 0, 贡献逐项为 0, 与精确路由代数等价 (fp32 最大相对差 1.4×10^{-6} , 浮点重结合级别)。

关键工程价值在于可编译性: 原 Triton grouped GEMM 对 dynamo 是黑盒, 每层 MoE 强制一次 graph break(每次推理 360 个), inductor 只能融合断点之间的碎片; 纯 torch 实现全程静态形状、无自定义内核, 可 fullgraph 编译, 使整步去噪成为一张图。设阈值开关 moe_dense_max_tokens=512: 去噪时 $T=51$ 走稠密, 大 T 回退 grouped 路径。

3.4 compile mode max-autotune-no-cudagraphs(commit 7eb22d83)

编译模式对比后选定 max-autotune-no-cudagraphs, 显著优于 default: 284.0 vs 336.6 ms/chunk(model-only)。

3.5 C4: preprocess GPU 化 + tokenizer 缓存 (commit 21ef4187)

关键发现: 把 HF processor 原样搬上 GPU 无效 (290 ms)——瓶颈不是图像算子本身, 而是 HF preprocess 的 Python 包装层 (kwargs 校验、group_images_by_shape、BatchFeature 等)。正解是 prepare_images_on_device: 在 GPU 上直接复刻同一组 torch 算子——tvF.resize(bicubic + antialias)、融合的 rescale/normalize、patchify, 以及逐行复制的 smart_resize 取整逻辑——跳过全部 Python 包装。单相机预处理 $236 \rightarrow 36.1$ ms。tokenizer 按 task 字符串缓存 (上限 16), 消除重复 encode。

3.6 C3: compile_prefix(commit 569929b8)

将 embed_prefix + 36 层 KV fill 整体 compile, model 再降 **65 ms** (-20%)。已知一次性 graph break(grid_thw 的 .item())、deepstack 按 layer_idx 特化, 需 recompile_limit=64 且 warmup ≥ 3 次。

3.7 fp16: 被否决的负结果

fp16 方案经实测后否决 (详见 §6): A100 上 fp16 与 bf16 tensor core 峰值相同, launch-bound 模型无收益 (294.7 vs 278.9 ms, 反而更慢), 且 LIBERO drift 3.33% 超过 2% 阈值 (bf16 为 0.66–0.73%)。

4 实验结果

4.1 推理: 优化路径 (SO101 单相机, 10 步去噪)

最终栈 (bf16 + dense MoE + max-autotune + compile_prefix + GPU 预处理) 相对 PR 基线: model-only **-72%**, e2e **-79%**。

4.2 推理: 与官方 upstream 对比 (robotwin 三相机 480×640)

同权重、同 workload、同污染环境窗口:

表 1: PR 侧推理优化路径 (A100,SO101 单相机, 污染环境同窗口测量)

阶段	配置	时延 (ms)
基线	PR 原版 (eager + fp32),e2e	1039.5
C1	sdpa+bf16 + 不变量外提 + sync-free MoE,e2e	951.9
C2	+ torch.compile predict_velocity,e2e	699.5
+ 稠密 MoE	双 GEMM 纯 torch MoE(fullgraph)	—
+ max-autotune	max-autotune-no-cudagraphs,model-only	284.0
最终栈	+ compile_prefix + GPU 预处理,e2e	≈300–315

表 2: 与官方 deploy 对比 (A100,robotwin 3 相机 480×640, 同权重同负载)

配置	预处理 (ms)	model (ms)	e2e (ms)
官方 eager	668.1	1020.7	1705.0
官方 use_compile	752.3	344.2	1113.0
本文优化栈	147.8	288.8	436.6

e2e 口径: 为官方 eager 的 **3.9×**, 为官方 use_compile 的 **2.55×**。注意官方路径在污染环境下 CPU 预处理波动大 (668–752 ms), 而本文 GPU 预处理受影响小。

4.3 训练吞吐 (robotwin 三相机 ckpt,fwd+bwd)

表 3: 训练单步对比 (fwd+bwd; 官方为 4×A100 fsdp2 + compile + grad_ckpt)

方案	配置	性能
官方 recipe	micro=2/卡,global=8,4 卡	2.999 s/step = 0.67 samples/s/GPU
PR 侧单卡 (原版)	B=2	2131.7 ms/step
PR 侧单卡 (优化后)	B=2	1049.8 ms/step(2.03×)
PR 侧单卡 (原版)	B=4	2762.6 ms/step
PR 侧单卡 (优化后)	B=4	1704.6 ms/step(1.62× , 峰值 62.6 GB)

官方稳态含 MoGe/DINO-video teacher 蒸馏前向;PR 移植版无此分支, 口径略有差异, 数字已注明。优化后 PR 单卡吞吐 2.35 samples/s, 约为官方每卡 (0.67) 的 **3.5×**。官方 recipe 在单卡上不可运行 (见 §2.3), 此处为各自可行配置下的对比。

4.4 保真度 (LIBERO open-loop drift, 阈值 2%)

全部落地项均低于 2% 阈值。GPU 预处理的张量级校验: 除 images 外全部 bit 一致;images 差 6.5×10^{-5} , 为 CUDA 与 CPU bicubic kernel 累积顺序差异。

5 与官方 upstream 对比小结

推理 e2e: 本文 436.6 ms vs 官方 eager 1705.0 ms(3.9×)、官方 compile 1113.0 ms(2.55×); 官方 130 ms @4090D claim 在 A100 上不可复现 (最优 compile model-only 344 ms)。训练: 官方依赖 32 卡 fsdp2 设计, 单卡 OOM; 本文 PR 侧单卡可行且每卡吞吐约 3.5×(口径: 官方含 teacher 蒸馏支线,PR 移植版无此分支)。

表 4: 保真度校验 (LIBERO 真实帧,rel L2 vs fp32/eager 参考, 固定 noise 种子)

检查项	drift
稠密 MoE on/off	0.76%
C4 全组合 (GPU 预处理等)	0.66%
C3 compile_prefix	0.69%
C2 max-autotune	0.73%
被否决:fp16	3.33%(超阈值)

6 讨论与局限

- **130 ms claim 为何无法在 A100 复现:** 本模型为 launch-bound 小模型, 其相对表现在 consumer 卡 (4090D) 与数据中心卡 (A100) 上差异显著——kernel 发射开销不随算力线性缩放。官方 claim 很可能是在该卡型特定条件下测得, 不宜跨卡型引用。
- **编译区内的 Python wrapper 是隐形杀手:**C4 的教训表明, 任何留在编译区内的 Python 包装层 (HF processor 式的 kwargs 校验、按形状分组等) 都会触发 per-layer graph break, 使 inductor 优化碎片化。“搬上 GPU” 不等于 “GPU 化”, 必须复刻算子序列、跳过包装。
- **fp16 负结果的价值:**A100 上 fp16/bf16 tensor core 同峰值,launch-bound 工况下换 dtype 无收益; 且 fp16 数值范围窄,drift 3.33% 超阈值。bf16 是正确默认。
- **局限:** ①所有绝对值测于污染宿主机 (load 21–57), 空闲机会更低, 但同窗口相对比较成立; ② LIBERO 无 lingbot 微调权重, 保真度校验是代理指标 (open-loop drift), 非闭环任务成功率, 部署前仍建议在目标平台上做闭环评测。

7 结论

在 “小 token、高频次、launch-bound” 的流匹配 VLA 工况下, 加速的正确次序是: 先消除 host 同步与循环不变量重复, 再让整步去噪成为单张可编译图 (纯 torch 稠密 MoE 是关键使能), 最后把预处理从 Python 包装层中解放到 GPU。最终 A100 上 e2e 从 1039.5 ms 降至约 300–315 ms, 训练单卡吞吐达官方 fsdp2 recipe 每卡的约 3.5 倍, 且全部优化 drift 不超过 0.76%。

参考与产物

[1] lerobot PR #3967: LingBot-VLA 2.0 policy(HuggingFace / Robbyant), 本文基线。

[2] Robbyant lingbot-vla-v2 upstream 仓库: 官方 deploy 路径与 130 ms @4090D 声明的对照实测。

[3] 优化分支 pr3967-combined:commits 4d692d25(C1)、b0544e9b(C2)、9de3cc41(dense MoE)、7eb22d83(max-autotune)、21ef4187(C4)、569929b8(C3)。

[4] 测量脚本 bench/bench_lingbot_v2.py; 姊妹报告 docs/accel/dense_moe_report.pdf(稠密 MoE 单项专题)。